Project README

Functionalities

- **User Authentication:** Secure login and registration system for users.
- **File Storage:** Upload and manage files using the MinIO object storage.
- **User Profiles:** View and edit user profiles, including avatars.
- **Posts and Comments:** Create, view, and delete posts.
- **Interactions:** Like and comment posts.
- **GraphQL:** The main service that handles all the application's functionalities interacts with the User's, Post's, and Interaction's services through GraphQL.

Installation and Setup

1. **Build the Project**

   Navigate to the project directory and run:

   ```
   mvn clean package -DskipTests
   ```

   This command cleans the project, compiles the source code, and packages it, skipping the tests.

2. **Start the Application with Docker Compose**

   ```
   docker compose up --build -d
   ```

   This command builds and starts all the services defined in the `docker-compose.yml` file in detached mode.

Running the Application

Once the Docker containers are up and running, the application should be accessible at:

- **Web Application URL:** `http://localhost:7000`

For testing purposes, the application comes with pre-configured user accounts:

Username	Password
user1	password1
user2	password2
user3	password3
user4	password4
user5	password5
user6	password6

You can log in using the usernames and passwords provided above, or create a new account.

> ***Note:*** *These accounts are for testing purposes only. Please do not use them in a production environment.*

Accessing the MinIO Dashboard

The MinIO Dashboard allows you to manage the application's object storage.

- **URL:** `http://localhost:9001`
- **Username:** `rootname`
- **Password:** `yourpassword`

After logging in, you can view, upload, and manage the files stored by the application.

Test GrpahQL API

> ***Note:*** *Access to the GraphQL API is restricted. You need to add to the requests an authorization header.*
>
> ```
> X-API-Key: yourapikey
> ```

You can test the GraphQL API by accessing the `/graphiql` endpoint (remember to add the auth header) or by sending a POST request to the `/graphql` endpoint of the different services:

- **User Service:** `http://user:7001/graphql`
  - The graphql schema configured in the service is:

```
type User {
 id: ID!
 username: String!
 email: String!
 password: String!
 avatarPath: String!
}

type Query {
 getUserById(id: ID!): User
 getUserByUsername(username: String!): User
 getUserByEmail(email: String!): User
getUsers: [User]!
}

type Mutation {
 createUser(username: String!, email: String!, password: String!, avatarPath:String!): User!
 deleteUser(id: ID!): Boolean
 updateUser(id: ID!, username: String, email: String, password: String, avatarPath: String): User
}
```

- **Post Service:** `http://post:7002/graphql`
  - The graphql schema configured in the service is:

```
type Post {
 id: ID!
 description: String!
 user: User!
 comments: [Comment]!
 likesCount: Int!
 imagePath: String!
}

type Query {
 getPostById(id: ID!): Post
 getPostsByUserId(userId: ID!): [Post]!
 getPosts: [Post]!
}

type Mutation {
 createPost(description: String!, userId: ID!, imagePath:String!): Post!
 deletePost(id: ID!): Boolean
}
```

- **Interaction Service:** `http://interaction:7003/graphql`
  - The graphql schema configured in the service is:

```
type Comment {
id: ID!
content: String!
postId: ID!
user: User!
}

type Like {
id: ID!
postId: ID!
userId: ID!
}

type Query {
getCommentsByPostId(postId: ID!): [Comment]
getLikesByPostId(postId: ID!): [Like]
isPostLikedByUser(postId: ID!, username: String!): Boolean
}

type Mutation {
addComment(content: String!, postId: ID!, userId: ID!): Comment
deleteComment(id: ID!): Boolean
likePost(postId: ID!, userId: ID!): Like
unlikePost(postId: ID!, userId: ID!): Boolean
}
```